

Trabalho Prático de

Matemática Discreta

2025/2026

Trabalho elaborado por:

Grupo (Trabalho individual)

8250670 Braima Djaguité único elemento do grupo

Curso de:

Licenciatura em Engenharia Informática

Docentes:

Eliana Costa e Silva (eos@estg.ipp.pt)

Isabel Cristina Duarte (icd@estg.ipp.pt)

Rosa Silveira (rmss@estg.ipp.pt)

Felgueiras, 5 de junho de 2026

Conteúdo

1	Introdução	4
2	Dados do Problema	4
3	Classificação do Grafo	7
3.1	Verificação do Teorema 1	8
4	Conexidade e Caminhos	9
5	Caminhos de Euler	10
6	Teoremas de Dirac e Ore	10
6.1	Verificação do Teorema de Dirac	10
6.2	Verificação do Teorema de Ore	11
7	Algoritmo de Dijkstra	11
7.1	Definição do Problema	11
7.2	Execução do Algoritmo	12
7.3	Resultado Final	12
8	Conclusão	13
9	Matriz de adjacência completa	13
10	Matriz de distância completa	14
11	Código utilizados	16
12	Prompts utilizados	24

Lista de Figuras

1	Representação do grafo obtido	6
---	---	---

Lista de Tabelas

1	Coordenadas tridimensionais dos vértices selecionados	5
2	Grau de cada vértice	8
3	Execução do algoritmo de Dijkstra	12

1 Introdução

O presente trabalho tem como objetivo a modelação e análise de um grafo tridimensional utilizando a linguagem Python e a biblioteca NetworkX.

A partir de um conjunto de coordenadas fornecidas, foi construído um grafo não orientado e ponderado, onde cada vértice representa um ponto do espaço e cada aresta representa uma ligação entre dois vértices. O peso de cada aresta corresponde à distância euclidiana entre os vértices ligados.

Após a construção do grafo, foram realizadas diversas análises, incluindo a obtenção das matrizes de adjacência e de distâncias, a classificação do grafo, a verificação da conectividade, a determinação da existência de caminhos eulerianos e hamiltonianos e a aplicação do algoritmo de Dijkstra para determinação do caminho mínimo entre dois vértices.

2 Dados do Problema

Foram selecionados 25 pontos tridimensionais do conjunto de coordenadas disponibilizado no enunciado.

Cada ponto foi representado por um vértice do grafo.

A criação das arestas foi efetuada com base na distância euclidiana entre pares de pontos. Duas posições foram ligadas por uma aresta sempre que a respetiva distância satisfazia a condição:

$$7 < d(i,j) < 10$$

sendo $d(i,j)$ a distância euclidiana entre os vértices i e j .

O peso associado a cada aresta corresponde à distância euclidiana entre os dois vértices ligados. Os vértices escolhidos foram estes que aparecem de seguida.

Vértice	x	y	z
1	0.0	10.0	5.0
2	5.0	5.0	0.0
3	10.0	0.0	5.0
4	5.0	-5.0	10.0
5	10.0	-10.0	15.0
6	5.0	-10.0	15.0
7	5.0	-5.0	5.0
8	5.0	-15.0	10.0
9	-5.0	5.0	5.0
10	0.0	5.0	5.0
11	10.0	-5.0	0.0
12	5.0	0.0	10.0
13	10.0	-15.0	5.0
14	-5.0	15.0	0.0
15	10.0	-5.0	15.0
16	10.0	-10.0	10.0
17	5.0	-10.0	5.0
18	0.0	5.0	0.0
19	10.0	10.0	0.0
20	5.0	-15.0	0.0
21	5.0	0.0	0.0
22	5.0	-15.0	15.0
23	10.0	15.0	5.0
24	-5.0	10.0	0.0
25	10.0	-5.0	5.0

Tabela 1: Coordenadas tridimensionais dos vértices selecionados

O critério que apliquei foi tentar escolher um conjunto de pontos que permitam formar um grafo que não tenha nenhum vértice isolado.

Matriz de Distâncias A matriz de distâncias D contém as distâncias euclidianas entre todos os pares de vértices. Devido à sua dimensão (25×25), apresenta-se apenas um excerto representativo. A matriz completa encontra-se no Anexo A.

$$D = \begin{bmatrix} 0.00 & 8.66 & 14.14 & 16.58 & 24.49 & \dots \\ 8.66 & 0.00 & 8.66 & 14.14 & 21.79 & \dots \\ 14.14 & 8.66 & 0.00 & 8.66 & 14.14 & \dots \\ 16.58 & 14.14 & 8.66 & 0.00 & 8.66 & \dots \\ 24.49 & 21.79 & 14.14 & 8.66 & 0.00 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Matriz de Adjacências A matriz de adjacências A representa as ligações existentes entre os vértices do grafo. O valor 1 indica a existência de uma aresta entre dois vértices e o valor 0 indica a ausência de ligação. Pelo mesmo motivo do caso anterior, apresenta-se apenas um excerto representativo. A matriz completa encontra-se no Anexo B.

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & \dots \\ 1 & 0 & 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & 1 & 0 & \dots \\ 0 & 0 & 1 & 0 & 1 & \dots \\ 0 & 0 & 0 & 1 & 0 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Apresente a representação Gráfica do Grafo

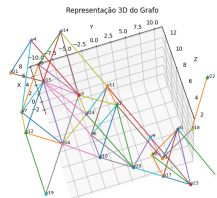


Figura 1: Representação do grafo obtido

3 Classificação do Grafo

A análise efetuada permitiu concluir que o grafo possui as seguintes características:

Grafo simples; Grafo não orientado; Grafo ponderado.

Foi também efetuado o cálculo dos graus de todos os vértices e também identificar o número de arestas.

$$|V| = 25$$

$$|E| = 53$$

Graus dos Vértices:

Vértice	Grau
0	5
1	4
2	6
3	7
4	3
5	4
6	5
7	5
8	3
9	4
10	4
11	5
12	4
13	1
14	4
15	9
16	7
17	4
18	2
19	2
20	6
21	2
22	1
23	4
24	5

Tabela 2: Grau de cada vértice

3.1 Verificação do Teorema 1

Para validar os resultados foi aplicado o Teorema Fundamental dos Grafos, segundo o qual a soma dos graus dos vértices deve ser igual ao dobro do número

de arestas. Ou seja, ao somar os graus de todos os vértices, o resultado deve ser igual ao dobro do número de arestas.

Os valores obtidos foram:

$$[\sum grau(v) = 106]$$

$$2|E| = 106$$

$$2 \cdot 53 = 106$$

$$106 = 106$$

Como ambas as quantidades coincidem, conclui-se que o teorema foi verificado.

4 Conexidade e Caminhos

Foi efetuada uma análise à conectividade do grafo.

Os resultados obtidos indicam que o grafo é conexo, ou seja, existe pelo menos um caminho entre qualquer par de vértices.

Adicionalmente foi determinado o número de caminhos simples entre um vértice inicial e final.

Vértice inicial: V0

Vértice final: v24

Número de caminhos encontrados:

$$\text{Número de caminho} = 260653$$

A verificação da conexidade foi realizada recorrendo à função de conectividade disponibilizada pela biblioteca NetworkX.

Para determinar o número de caminhos simples entre o vértice inicial e o vértice final foi utilizado o algoritmo de enumeração de caminhos simples.

5 Caminhos de Euler

Verificar as condições de existência de:

- Caminho de Euler
- Circuito de Euler

Um caminho de Euler existe apenas quando o grafo possui exatamente zero ou dois vértices de grau ímpar. Um Circuito de Euler existe apenas quando o grafo possui exatamente zero vértices de grau ímpar.

Após o cálculo dos graus dos vértices verificou-se a existência de 12 vértices de grau ímpar.

Como o número de vértices de grau ímpar é superior a dois, conclui-se que não existe caminho de Euler. E, como o número de vértices de grau ímpar é superior a zero, conclui-se que não existem circuitos de Euler. Na verdade, bastava verificar a não existência dos caminhos de Euler para, consequentemente, confirmar a não existência dos circuitos de Euler.

6 Teoremas de Dirac e Ore

6.1 Verificação do Teorema de Dirac

O Teorema de Dirac afirma que um grafo simples com (n) vértices é Hamiltoniano se todos os seus vértices tiverem grau pelo menos igual a $(\frac{n}{2})$. Para o grafo em estudo:

$$[n = 25]$$

$$\frac{n}{2} = 12.5$$

Verificando os graus dos vértices, observa-se que o maior grau é 9 e que vários vértices possuem grau inferior a 12.5.

Assim, a condição

$$\deg(v) \geq \frac{n}{2}$$

não é satisfeita para todos os vértices.

Conclusão: O grafo não satisfaz o Teorema de Dirac. Por isso, não é possível garantir a existência de um ciclo Hamiltoniano com base neste teorema.

6.2 Verificação do Teorema de Ore

O Teorema de Ore afirma que um grafo simples com (n) vértices é Hamiltoniano se, para todos os pares de vértices não adjacentes (u) e (v) , se verificar:

$$[deg(u) + deg(v) \geq n]$$

Para o grafo em estudo:

$$[n = 25]$$

Foi analisada a soma dos graus dos pares de vértices não adjacentes, verificando-se a existência de pares para os quais:

$$[deg(u) + deg(v) < 25]$$

Assim, a condição do Teorema de Ore não é satisfeita para todos os pares de vértices não adjacentes.

Conclusão: O grafo não satisfaz o Teorema de Ore. Por isso, não é possível garantir a existência de um ciclo Hamiltoniano com base neste teorema.

7 Algoritmo de Dijkstra

7.1 Definição do Problema

Pretende-se determinar o caminho de custo mínimo entre um vértice inicial e outro vértice final do grafo ponderado fornecido. O grafo é não orientado e ponderado, sendo o peso de cada aresta igual à distância euclidiana entre os vértices que ela liga. Assim, o objetivo consiste em encontrar a sequência de vértices que permite ligar o vértice inicial ao vértice final ao final minimizando a soma dos pesos das arestas percorridas.

Vértice inicial: V0

Vértice final: V24

7.2 Execução do Algoritmo

O algoritmo de Dijkstra foi executado considerando o vértice v_0 como origem e o vértice v_{24} como destino. Em cada iteração é selecionado o vértice com menor distância temporária e são atualizadas as distâncias dos seus vértices adjacentes.

Iteração	Nó Atual	Distâncias Atualizadas	Predecessor
0	v_0	$v_1 = 8.66, v_8 = 7.07, v_{13} = 8.66, v_{17} = 7.07, v_{23} = 7.07$	—
1	v_8	Nenhuma atualização	v_0
2	v_{17}	$v_{20} = 14.14$	v_0
3	v_{23}	$v_9 = 15.73$	v_0
4	v_1	$v_2 = 17.32, v_{18} = 15.73$	v_0
5	v_{13}	Nenhuma atualização	v_0
6	v_{20}	$v_6 = 21.21, v_{10} = 21.21, v_{24} = 22.80$	v_{17}
7	v_9	$v_{11} = 24.39$	v_{23}
8	v_{18}	$v_{22} = 22.80$	v_1
9	v_2	$v_3 = 25.98$	v_1
10	v_6	$v_{15} = 29.87$	v_{20}
11	v_{10}	$v_{16} = 29.87$	v_{20}
12	v_{24}	Destino alcançado	v_{20}

Tabela 3: Execução do algoritmo de Dijkstra

7.3 Resultado Final

Distância mínima:

$$d = 22.80$$

Caminho mínimo:

$$v_0 \rightarrow v_{17} \rightarrow v_{20} \rightarrow v_{24}$$

8 Conclusão

Neste trabalho foi efetuada a construção e análise de um grafo ponderado a partir de um conjunto de coordenadas tridimensionais.

Foram geradas as matrizes de representação do grafo, determinada a sua classificação, analisada a conectividade e verificadas propriedades relacionadas com caminhos eulerianos e hamiltonianos.

Os resultados obtidos permitiram concluir que o grafo é simples, não orientado, ponderado e conexo. Foi igualmente verificado o Teorema Fundamental dos Grafos e demonstrada a inexistência de caminho de Euler. Aplicando os teoremas de DIRAC e ORE, não foi possível garantir que o grafo é Hamiltoniano.

Por fim, através da aplicação do algoritmo de Dijkstra, foi determinado o caminho mínimo entre os vértices v_0 e v_{24} , obtendo-se um custo total de 22.80 unidades.

Os objetivos definidos para o trabalho foram todos alcançados.

9 Matriz de adjacência completa

```
[[0 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1 0]
 [1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0]
 [0 1 0 1 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 1]
 [0 0 1 0 1 1 0 0 0 0 0 0 0 1 1 1 0 0 0 0]
 [0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0]
 [0 0 0 1 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 0]
 [0 0 1 0 0 0 0 0 0 0 1 1 0 0 0 1 0 0 0 0]
 [0 0 0 0 1 1 0 0 0 0 0 0 1 0 0 1 1 0 0 0]
 [1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0]
 [0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0]
 [0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0]
 [0 0 1 0 0 0 1 0 0 1 0 0 0 0 1 0 0 0 0 0]
 [0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 1 0 0 1 0]
 [1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]]
```

```
[0 0 0 1 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 1 0 1 1 1 0 0 0 0 1 0 1 0 1 0 0 0 0 1 0 0 1]
[0 0 0 1 0 0 0 1 0 0 1 0 1 0 0 1 0 0 0 1 0 0 0 0 1]
[1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 1 0 0 0 1 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1]
[0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[1 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 0 0 0 1 0 0 0 0]
```

10 Matriz de distância completa

```
[[ 0.  8.66 14.14 16.58 24.49 22.91 15.81 25.98 7.07 5.  18.71 12.25 26.93 8.66
20.62 22.91 20.62 7.07 11.18 25.98 12.25 27.39 11.18 7.07 18.03]
```

```
[ 8.66 0.  8.66 14.14 21.79 21.21 11.18 22.36 11.18 7.07 11.18 11.18 21.21 14.14
18.71 18.71 15.81 5.  7.07 20.  5.  25. 12.25 11.18 12.25]
```

```
[14.14 8.66 0.  8.66 14.14 15.  7.07 16.58 15.81 11.18 7.07 7.07 15.  21.79 11.18
11.18 11.18 12.25 11.18 16.58 7.07 18.71 15.  18.71 5.  ]
```

```
[16.58 14.14 8.66 0.  8.66 7.07 5.  10.  15.  12.25 11.18 5.  12.25 24.49 7.07 7.07
7.07 15.  18.71 14.14 11.18 11.18 21.21 20.62 7.07]
```

```
[24.49 21.79 14.14 8.66 0.  5.  12.25 8.66 23.45 20.62 15.81 12.25 11.18 32.79 5.
5.  11.18 23.45 25.  16.58 18.71 7.07 26.93 29.15 11.18]
```

```
[22.91 21.21 15.  7.07 5.  0.  11.18 7.07 20.62 18.71 16.58 11.18 12.25 30.82 7.07
7.07 10.  21.79 25.5 15.81 18.03 5.  27.39 26.93 12.25]
```

```
[15.81 11.18 7.07 5.  12.25 11.18 0.  11.18 14.14 11.18 7.07 7.07 11.18 22.91 11.18
8.66 5.  12.25 16.58 11.18 7.07 14.14 20.62 18.71 5.  ]
```

```
[25.98 22.36 16.58 10.  8.66 7.07 11.18 0.  22.91 21.21 15.  15.  7.07 33.17 12.25
7.07 7.07 22.91 27.39 10.  18.03 5.  30.82 28.72 12.25]
```

[7.07 11.18 15.81 15. 23.45 20.62 14.14 22.91 0. 5. 18.71 12.25 25. 11.18 20.62 21.79 18.03 7.07 16.58 22.91 12.25 24.49 18.03 7.07 18.03]

[5. 7.07 11.18 12.25 20.62 18.71 11.18 21.21 5. 0. 15. 8.66 22.36 12.25 17.32 18.71 15.81 5. 12.25 21.21 8.66 22.91 14.14 8.66 14.14]

[18.71 11.18 7.07 11.18 15.81 16.58 7.07 15. 18.71 15. 0. 12.25 11.18 25. 15. 11.18 8.66 14.14 15. 11.18 7.07 18.71 20.62 21.21 5.]

[12.25 11.18 7.07 5. 12.25 11.18 7.07 15. 12.25 8.66 12.25 0. 16.58 20.62 8.66 11.18 11.18 12.25 15. 18.03 10. 15.81 16.58 17.32 8.66]

[26.93 21.21 15. 12.25 11.18 12.25 11.18 7.07 25. 22.36 11.18 16.58 0. 33.91 14.14 7.07 7.07 22.91 25.5 7.07 16.58 11.18 30. 29.58 10.]

[8.66 14.14 21.79 24.49 32.79 30.82 22.91 33.17 11.18 12.25 25. 20.62 33.91 0. 29.15 30.82 27.39 11.18 15.81 31.62 18.03 35. 15.81 5. 25.5]

[20.62 18.71 11.18 7.07 5. 7.07 11.18 12.25 20.62 17.32 15. 8.66 14.14 29.15 0. 7.07 12.25 20.62 21.21 18.71 16.58 11.18 22.36 25.98 10.]

[22.91 18.71 11.18 7.07 5. 7.07 8.66 7.07 21.79 18.71 11.18 11.18 7.07 30.82 7.07 0. 7.07 20.62 22.36 12.25 15. 8.66 25.5 26.93 7.07]

[20.62 15.81 11.18 7.07 11.18 10. 5. 7.07 18.03 15.81 8.66 11.18 7.07 27.39 12.25 7.07 0. 16.58 21.21 7.07 11.18 11.18 25.5 22.91 7.07]

[7.07 5. 12.25 15. 23.45 21.79 12.25 22.91 7.07 5. 14.14 12.25 22.91 11.18 20.62 20.62 16.58 0. 11.18 20.62 7.07 25.5 15. 7.07 15.]

[11.18 7.07 11.18 18.71 25. 25.5 16.58 27.39 16.58 12.25 15. 15. 25.5 15.81 21.21 22.36 21.21 11.18 0. 25.5 11.18 29.58 7.07 15. 15.81]

[25.98 20. 16.58 14.14 16.58 15.81 11.18 10. 22.91 21.21 11.18 18.03 7.07 31.62 18.71 12.25 7.07 20.62 25.5 0. 15. 15. 30.82 26.93 12.25]

[12.25 5. 7.07 11.18 18.71 18.03 7.07 18.03 12.25 8.66 7.07 10. 16.58 18.03 16.58 15. 11.18 7.07 11.18 15. 0. 21.21 16.58 14.14 8.66]

[27.39 25. 18.71 11.18 7.07 5. 14.14 5. 24.49 22.91 18.71 15.81 11.18 35. 11.18 8.66 11.18 25.5 29.58 15. 21.21 0. 32.02 30.82 15.]

[11.18 12.25 15. 21.21 26.93 27.39 20.62 30.82 18.03 14.14 20.62 16.58 30. 15.81 22.36 25.5 25.5 15. 7.07 30.82 16.58 32.02 0. 16.58 20.]

```
[ 7.07 11.18 18.71 20.62 29.15 26.93 18.71 28.72 7.07 8.66 21.21 17.32 29.58 5.
25.98 26.93 22.91 7.07 15. 26.93 14.14 30.82 16.58 0. 21.79]

[18.03 12.25 5. 7.07 11.18 12.25 5. 12.25 18.03 14.14 5. 8.66 10. 25.5 10. 7.07
7.07 15. 15.81 12.25 8.66 15. 20. 21.79 0. ]]
```

11 Código utilizados

```
import math

import networkx as nx

import numpy as np

from mpl_toolkits.mplot3d import Axes3D

import matplotlib.pyplot as plt

=====
FUNÇÃO DISTÂNCIA 3D =====

def calcular_distancia(p1, p2) :

    x1, y1, z1 = p1
    x2, y2, z2 = p2

    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2 + (z2 - z1)**2)

=====
CONSTRUÇÃO DO GRAFO =====

def criar_grafo(pontos) :

    G = nx.Graph()

    adicionar_vértices for i in range(len(pontos)): G.add_node(i, pos = pontos[i])

    adicionar_arestas_válidas for i in range(len(pontos)):

        for j in range(i + 1, len(pontos)):

            distancia = calcular_distancia(pontos[i], pontos[j])

            if 7 ≤ distancia ≤ 10:

                G.add_edge(i, j, weight = round(distancia, 2))

    return G
```

```

=====
MATRIZES =====
def gerar_matrizes(G, pontos) :
    n = len(pontos)

    matriz de adjacência matriz_adj = nx.to_numpy_array(G, nodelist = range(n), weight =
    None)

    matriz de distâncias matriz_dist = np.zeros((n, n))

    for i in range(n):
        for j in range(n):
            if i != j:
                matriz_dist[i][j] = round(calcular_distancia(pontos[i], pontos[j]), 2)

    return matriz_adj, matriz_dist

=====
Imprimir matrizes =====
def imprimir_matrizes(matriz_adj, matriz_dist) :
    print("===== ALÍNEA A =====")

    np.set_printoptions(linewidth = 200, threshold = np.inf)

    print("MATRIZ DE ADJACÊNCIA:") print(matriz_adj.astype(int))

    print("DE DISTÂNCIAS:") print(np.round(matriz_dist, 2)) =====
    Gerararepresentaçãográficadografo =====
    defdesenhar_grafo3d(G) :

        fig = plt.figure(figsize=(12, 10)) ax = fig.add_subplot(111, projection = '3d')

        posicoes = nx.get_node_attributes(G, 'pos')

        desenhar vértices for v, (x, y, z) in posicoes.items():

            ax.scatter(x, y, z)

            ax.text( x, y, z, f'v{v}', fontsize=8 )

        desenhar arestas for u, v in G.edges():

            x = [posicoes[u][0], posicoes[v][0]] y = [posicoes[u][1], posicoes[v][1]] z = [posi-
            coes[u][2], posicoes[v][2]]

```

```

ax.plot(x, y, z)

ax.set_title("Representação 3D do Grafo")

ax.set_xlabel("X")ax.set_ylabel("Y")ax.set_zlabel("Z")

plt.savefig("grafo3d.png", dpi = 300, bbox_inches = "tight")

plt.show()

```

=====

ALÍNEA B CLASSIFICAÇÃO, GRAUS E TEOREMA 1 =====

```

def analisar_grafo(G, matriz_adj) :

print("===== ALÍNEA B =====")

classificação

print("Classificação do grafo:") print(- Simples") print(- Não orientado") print(-
Ponderado")

n = len(matriz_adj)

graus = []

print("dos vértices:")

for i in range(n):

grau = int(sum(matriz_adj[i]))graus.append(grau)

print(f"vi: grau")

numero_arestas = G.number_of_edges()

soma_graus = sum(graus)

print(f"úmero de arestas = numero_arestas")

print("ção do Teorema 1:")

print(f"Soma dos graus = soma_graus")print(f" $2x|E| = 2 * numero_arestas$ ")

if soma_graus == 2 * numero_arestas : print("Teorema1verificado.")else :
print("Teorema1NÃOverificado.")

```

=====

ALÍNEA C CONEXIDADE E CAMINHOS =====

```

def analisar_conexidade(G, origem, destino) :

```

```

print("=====") print("ALÍNEA
C") print("=====")

if nx.is_connected(G) :
    print("grafo é conexo.")
else:
    print("grafo é desconexo.")

componentes = list(nx.connected_components(G))

print(f"Número de componentes conexas: len(componentes)")

try:
    caminhos = list( nx.all_simple_paths(G, source = origem, target = destino))
    print( f"úmero de caminhos simples entre "f"origem e destino: len(caminhos)")
except nx.NetworkXNoPath:
    print( "ão existe caminho entre origem e destino.")

=====
ALÍNEA D CAMINHOS DE EULER =====

def verificar_euler(G) :

    print("=====") print("ALÍNEA
D") print("=====")

    graus = dict(G.degree())

    impares = []

    for vertice, grau in graus.items():
        if grau

    impares.append(vertice)

    print( f"úmero de vértices com grau ímpar: "f"len(impares)")

    if len(impares) == 0:
        print("Existe circuito de Euler.")

    elif len(impares) == 2:
        print("Existe caminho de Euler.")

```

else:

print("Não existe caminho de Euler.")

=====

ALÍNEA E ORE E DIRAC =====

def verificar_{hamiltoniano}(G) :

print("===== ALÍNEA E =====")

n = G.number_{of}nodes()

graus = dict(G.degree())

DIRAC

dirac = True

for grau in graus.values():

if grau % n / 2: dirac = False break

if dirac: print("Teorema de Dirac verificado.") else: print("Teorema de Dirac não verificado.")

ORE

ore = True

for u in G.nodes():

for v in G.nodes():

if u != v and not G.has_{edge}(u, v) :

if graus[u] + graus[v] % n:

ore = False break

if not ore: break

if ore: print("Teorema de Ore verificado.") else: print("Teorema de Ore não verificado.")

=====

ALÍNEA F DIJKSTRA (FORMATO DOS SLIDES) =====

def nome(v): return f"v{v}"

def dijkstra(G, origem, destino):

```

print("===== ALÍNEA F =====")

M = set()

X = [] Xd = [] R = []

vd = origem Mc = [origem]

print( f''It':;4''vd(M)';;20''Mc';;20''A';;20''vi,...,vd,vj e L(vj)';;50''X';;20''Xd';;20''R''')

print("-"* 220)

it = 0

while True:

    ----- determinar A -----

    A = []

    for vizinho in G.neighbors(vd):

        if vizinho not in M:

            A.append(vizinho)

    calculos = []

    ----- atualizar X, Xd e R -----

    custovd = 0

    if it != 0:

        for i in range(len(R)):

            if R[i] == Mc:

                custovd = Xd[i]break

        for vizinho in A:

            peso = G[vd][vizinho]["weight"]

            if it == 0:

                novocusto = peso

            else:

                caminhovd = Mc[: -1]

                custovd = 0

```

```

for i in range(len(R)):
    if R[i] == caminho_vd :
        custo_vd = Xd[i]break
    novo_custo = (sum(G[Mc[k]][Mc[k+1]][weight])forkinrange(len(Mc)-1)) +
    peso)
    novo_aminho = Mc + [vizinho]
    calculos.append(f'it:{i}f'.join(nome(v) for v in novo_aminho)f- > L(nome(vizinho)) =
    novo_custo : .2f")
    if vizinho in X:
        pos = X.index(vizinho)
        if novo_custo < Xd[pos] :
            Xd[pos] = novo_custoR[pos] = novo_aminho
        else:
            X.append(vizinho) Xd.append(novo_custo)R.append(novo_aminho)
    _____ imprimir linha atual _____
    texto_m = (" " + " , ".join(nome(v)forvinsorted(M))+)
    texto_a = (" " + " , ".join(nome(v)forvinA)+)
    texto_x = (" " + " , ".join(nome(v)forvinX)+)
    texto_xd = (" " + " , ".join(f"d : .2f"fordinXd)+)
    texto_r = "; ".join(- > ".join(nome(v)forvincaminho)forcaminho inR)
    print( f'it:{i}f'(nome(vd)+' '+texto_m) :< 20" f"'- >' .join(nome(v)forvinMc) :<
    20" f"texto_a :< 20" f"' ;' .join(calculos) :< 50" f"texto_x :< 20" f"texto_xd :<
    20" f"texto_r")
    _____ adicionar vd a M _____
    M.add(vd)
    _____ remover vd de X caso exista _____
    _____
    if vd in X:

```

```

pos = X.index(vd)

X.pop(pos) Xd.pop(pos) R.pop(pos)

_____ condição de paragem _____

if vd == destino:
    print("MÍNIMO:")
    print( -i_ ".join( nome(v) for v in Mc ) )
    distanciatotal = sum(G[Mc[k]][Mc[k+1]]["weight"] for k in range(len(Mc)-1))
    print( f"Distância total = distanciatotal : .2f" )
    return

if len(X) == 0:
    print("Não existe caminho.") return

_____ escolher novo vd _____

indicemin = Xd.index(min(Xd))
vd = X[indicemin] Mc = R[indicemin]

it += 1 =====
MAIN =====
def main():

    pontos = [ (0.0, 10.0, 5.0), (5.0, 5.0, 0.0), (10.0, 0.0, 5.0), (5.0, -5.0, 10.0), (10.0,
-10.0, 15.0), (5.0, -10.0, 15.0), (5.0, -5.0, 5.0), (5.0, -15.0, 10.0), (-5.0, 5.0, 5.0),
(0.0, 5.0, 5.0), (10.0, -5.0, 0.0), (5.0, 0.0, 10.0), (10.0, -15.0, 5.0), (-5.0, 15.0,
0.0), (10.0, -5.0, 15.0), (10.0, -10.0, 10.0), (5.0, -10.0, 5.0), (0.0, 5.0, 0.0), (10.0,
10.0, 0.0), (5.0, -15.0, 0.0), (5.0, 0.0, 0.0), (5.0, -15.0, 15.0), (10.0, 15.0, 5.0),
(-5.0, 10.0, 0.0), (10.0, -5.0, 5.0) ]

    G = criar_grafo(pontos) matrizadj, matrizdist = gerar_matrizes(G, pontos) desenhar_grafo3d(G)

    imprimir_matrizes(matrizadj, matrizdist)

    print("Ceguei aqui");

    analisar_grafo(G, matrizadj)

    analisar_conexidade(G, origem = 0, destino = 24)

    verificar_uler(G)

```

```
verificar_hamiltoniano(G)
```

```
dijkstra(G, origem=0, destino=24)
```

```
if __name__ == "__main__":main()
```

12 Prompts utilizados

Durante a elaboração deste trabalho foi utilizada uma ferramenta de Inteligência Artificial (ChatGPT) como apoio à compreensão do funcionamento das bibliotecas python usados, validação da implementação e preparação do relatório. Seguem-se os principais prompts utilizados:

1. "Explicar sobre as funções disponibilizadas pela biblioteca networkx."
2. "Analisar a implementação das funções de criar grafo, matrizes, análise da conectividade, de caminhos de euler e teoremas de DIRAC e ORE."
3. "Explicar sobre como implementar o algoritmo de Dijkstra"
4. "Analisar a implementação do algoritmo de Dijkstra em Python e verificar a correspondência entre as estruturas M, X, Xd e R utilizadas nos slides da disciplina e as estruturas implementadas no código."
5. "Identificar possíveis inconsistências entre a tabela gerada pelo programa e a execução do algoritmo apresentada nos exemplos fornecidos pelo docente."
6. "Corrigir a implementação do algoritmo de Dijkstra para que a tabela produzida siga a mesma lógica e notação utilizada nos slides da unidade curricular."
7. "Gerar o código que gere a representação gráfica do grafo."
8. "Explicar o funcionamento da biblioteca matplotlib."
9. "Adaptar o texto do relatório ao modelo de relatório fornecido pelo docente."
10. "Redigir as secções Introdução, Definição do Problema, Execução do Algoritmo e Conclusão."
11. "Auxiliar na construção de tabelas em LaTeX para apresentar a execução do algoritmo de Dijkstra."
12. "Rever a coerência entre os resultados obtidos pelo programa, o caminho

mínimo encontrado e as conclusões apresentadas no relatório.”

A ferramenta foi utilizada exclusivamente como apoio à análise, compreensão das coisas não compreendidas, interpretação dos resultados e preparação do relatório, sendo a implementação, execução dos testes e validação final dos resultados realizadas pelo autor, no caso, eu.